

Design of Digital Circuits—A Systems Approach

LÄSANVISNINGAR

Rolf Johannesson

Varför digitalteknik?

Vi rör oss ständigt i en värld full av digitaltekniska apparater. Kanske tänker vi i första hand på datorer, mobiltelefoner, CD-spelare, internetförbindelser etc, men även apparater som mikrovågsugnar och diskmaskiner är numera ”digitaliserade”. Mindre uppenbart är att det krävs ett digitaltekniskt synsätt, som vi förmedlar i denna kurs, för att konstruera exempelvis trafikljus för gatukorsningar. Fundera gärna en stund över hur du skulle konstruera ett sådant! Styranordningen skall tända och släcka signallamporna enligt givna sekvenser som dessutom påverkas av insignaler från givare i gatan eller från fotgängares tryckknappar. För att strukturera problemet behöver vi veta var i en signalsekvens vi befinner oss och vad som skall hända härnäst. I kapitel 2 kommer vi att införa ett begrepp som vi kallar *tillstånd* som är ett kondensat av vad som hänt hittills. Detta betraktelsesätt är egentligen en förutsättning för att vi på ett effektivt sätt skall kunna lösa en vid klass av problem. Då du läst igenom kapitel 2 kan du gärna återvända till trafik Korsningen och fundera vidare över ett angreppssätt. I laborationskursen kommer du att konstruera en styranordning för ett trafikljus.

Allmänt

Den dramatiska utvecklingen inom mikroelektroniken har bl a resulterat i rutinmässig konstruktion av kretsar med hög komplexitet. Konstruktionshjälpmedel som tidigare försvarade sin plats i läroböcker är nu passé. Då vi började arbetet med att revidera kurslitteraturen kastade vi helt sonika ut Quine-McCluskeys klassiska metod (från 1950-talet) för förenkling av kombinationskretsar – den fungerar helt enkelt inte för dagens komplexa konstruktioner! Vi reducerade tillståndspartitioneringsmetoden för tillståndskodning (från 1970-talet) till ett minimum. Metoden är elegant för läroboksexempel men senare forskning har visat att för realistiskt stora exempel ger metoden inga komplexitetsvinster. Under LTH’s första 40 år ägnades mycket exercerande åt konstruktion med så kallade SR-, JK- och T-vippor. Numera är dessa vippor knappast ens av historiskt intresse. Därför har de självfallet utgått ur kursen. Därigenom har vi skapat ökat utrymme för mer övning i att ”översätta” problem beskrivna med text till tillståndsgrafer. Detta är kursens viktigaste moment! Vidare har vi förstärkt avsnitten om algebraiska strukturer och de praktiskt viktiga lineära sekvensnäten. Därigenom får teknologerna gedigna generella kunskaper som kommer att vara konkurrenskraftiga under relativt lång tid.

En fördel med upplägget är att kursen inte kräver några specifika förkunskaper, men väl en viss mognad och abstrakt tänkande. Samtidigt kan abstraktionsnivån – åtminstone i början – upplevas svår vilket motverkas genom övning. Dessvärre upplever många teknologer abstraktioner och bevis som onödigt mumbo jumbo – det är de inte! På denna punkt måste ni lita på vår 20–30 års långa erfarenhet. Ett bevis kan förtjäna sin plats i en bok även om ingen teknolog lär sig det (vid en första genomläsning)!

Varning!

En stor nackdel med upplägget är att innehållet kanske inte motsvarar teknologernas förväntningar – särskilt inte om de läst en kurs i digitalteknik på lägre nivå! Därför utfärdar vi en varning för en abstraktionsnivå som vi emellertid är övertygade om är önskvärd och som man med större erfarenhet lär sig att uppskatta. Dessa läsanvisningar är förhoppningsvis en hjälp på vägen.

Matematiska förkunskaper

I likhet med praktiskt taget all ingenjörslitteratur använder vi begrepp från *mängdläran*. Läs därför först igenom Appendix A där vi sammanfattar några av de viktigaste beteckningarna och sambanden.

Kapitel 1

I Kap. 1 diskuteras först begreppet *matematisk modell*. Den holländske reglerteknikern och matematikern Jan Willems introducerade på 1980-talet en matematisk modell som ett par av mängder $(\mathcal{U}, \mathcal{B})$ som består dels av mängden av *utfall* (*outcomes*) $\mathcal{U}$ och dels av delmängden $\mathcal{B} \subseteq \mathcal{U}$ som kallas *beteende* (*behavior*). Den matematiska modellen specificerar beteendet, dvs de möjliga utfallen.

Matematiska modeller används mycket inom såväl ingenjörsvetenskaperna som andra vetenskaper som biologi, ekonomi, fysik, kemi etc. I gymnasiet har du använt matematiska modeller – kanske utan att du varit medveten om det – exempelvis då du räknat på en hävstång eller tillämpat Ohms lag på ett elläreproblem.

Kapitlet innehåller exempel på matematiska modeller från olika områden.

Vi kallar matematiska modeller vars beteenden är tidsfunktioner, dvs signaler som varierar med tiden, för *dynamiska system*. I Exempel 1.4 betraktar vi planetrörelserna som ett dynamiskt system vars beteende är de planetbanor som uppfyller Keplers lagar!

Tycker du att det är svårt att tillgodogöra dig innehållet i Kap. 1 så hoppa över det vid första genomläsningen och gå direkt till Kap. 2!

Kapitel 2

Detta är det viktigaste kapitlet i boken och det viktigaste avsnittet i kursen! Vi utnyttjar *tillståndsgrafer*, som ett mycket kraftfullt hjälpmedel då vi skall beskriva dynamiska system. Tillståndsgraferna bygger på att vi inför *tillstånd*, som delar upp beteendet i det som har skett och det som kommer att ske.

Tillstånd kan introduceras på olika sätt. Vanligast är att man ("drar fram dem ur trollkarlens hatt och") slår fast att:

Tillståndet är en kompakt beskrivning av förutvarande aktiviteter i systemet som tillsammans med nuvarande insignal är tillräcklig för att bestämma nuvarande utsignal och nästa tillstånd.

Tänk dig att ett system befinner sig i ett visst tillstånd *vid* tiden t_0 och att vi känner insignalen endast *från och med* tiden t_0 . Dessa uppgifter räcker för att vi skall kunna bestämma utsignalen *från och med* tiden t_0 ! *Hur* vi uppnådde tillståndet vid tiden t_0 spelar ingen roll för beteendet *från och med* denna tidpunkt. Detta följer direkt ur definitionen av tillstånd och är anledningen till att tillståndsgrafer är kraftfulla och viktiga hjälpmedel i många ingenjörsämnen!

Efter definitionen av tillstånd kan du gå direkt till avsnitt 2.2 (Finite-state machines) och studera exemplen ingående!

Jag rekommenderar emellertid att du (åtminstone delvis) följer upplägget i avsnitt 2.1. Framställningen bygger på att alla vet hur man räknar set i en tennismatch. Vi betraktar en följd av bäst-av-tre-set-matcher. Beteendet består av en sekvens (följd) av insignal-, utsignal-par, $\dots$, (b, B) , (a, C) , (b, C) , (a, A) , (a, C) , $\dots$ där exempelvis (b, B) utläses:

insignal b = "B vinner setet"
utsignal B = "B vinner matchen"
utsignal C = "Matchen fortsätter"

En sådan sekvens beskriver det dynamiska skeendet i följderna av matcherna och kan illustreras grafiskt med en kurva (trajektoria); se Fig. 2.1. I denna figur visar vi fem olika beteenden—fem olika resultatföljder. Betraktar vi dessa fem kurvor en stund så ser vi att de två första kurvorna kan slås ihop eftersom de är identiska fram till och med tiden t_0 . Samma sak gäller för de tredje och fjärde kurvorna. Se Fig. 2.2. De båda par av kurvor som vi skapat har samma beteende för tiden från och med t_0 . Alltså kan vi slå ihop dessa par enligt Fig. 2.3.

Låt oss nu antaga att vi i stället för bara fem kurvor har alla tänkbara beteenden (oändligt många!) och slår ihop dem på samma sätt. Då får vi fyra knippen av kurvor enligt Fig. 2.4. Efter en stunds eftertanke kommer vi fram till att de fyra kurvknippen motsvarar set-ställningarna: "0 – 0", "1 – 0", "0 – 1", "1 – 1" vid tidpunkten t_0 .

Upprepa denna procedur för alla tider; alltså inte bara tidpunkten t_0 ! Då får vi kurvorna i Fig. 2.8. De bildar ett spaljéliknande mönster. Därför kallas en sådan struktur (graf) för *trellis* (spaljé). All information finns samlad i denna "tennis-trellis"!

Exempel: Om det står "0 – 1" i matchen och A vinner setet, så går vi från punkten "0 – 1" snett nedåt längs linjen a/C och kommer till punkten "1 – 1" en tidsenhet senare; a svarar mot *nuvarande insignal* "A vinner setet" och C mot *nuvarande utsignal* "Matchen fortsätter" (eftersom A kvitterar till "1 – 1" måste de spela skiljeset!)

Trellisstrukturer dyker upp i många olika tillämpningar, till exempel inom signalbehandling, modulation, felkorrigering och ingår i dubbel uppsättning i GSM-telefoner.

Då vi betraktar en trellis så låter vi tiden löpa från vänster till höger. Alternativt kan vi låta tidsaxeln kollapsa och får då en struktur som visas i Fig. 2.9, nämligen en *tillståndsgraf*.

Vi har nu på ett rättframt och naturligt sätt visat hur vi kan komma från följder av insignal-, utsignal-par via trellisar till tillståndsgrafer!

Sambandet tillståndsgraf-trellis illustreras i Exempel 2.2 och 2.3. Det är viktigt att du förstår detta samband!

Studera Exempel 2.4 (Lejonburen) ingående. Följ vad som händer i tillståndsgrafen då exempelvis ett lejon ångrar sig och backar då han befinner sig mitt framför givarna. Övertyga dig om det inte går att lösa problemet med endast tre tillstånd.

Studera Exempel 2.5–2.9 samt räkna många av övningstalen som hör till detta kapitel. Det är viktigt att du lär dig att från en verbal eller matematisk beskrivning av ett problem kunna rita en tillståndsgraf. Detta är definitivt det viktigaste momentet i kursen! Med en lätt överdrift kan man påstå att återstoden av konstruktionsarbetet kan automatiseras.

Framställningen att införa tillstånd via trajektorior (kurvor) bygger på Jan Willems arbeten. Dessa rön te sig stor uppmärksamhet i vetenskapliga kretsar under 90-talet.

I avsnitt 2.3 (Sequential circuits) visar vi hur vi kan sätta samman enkla digitaltekniska *moduler* till nät, som kallas *sekvensnät* (*sequential circuits*), med samma beteenden som tillståndsgrafer. Därigenom har vi visat hur vi kan gå från en verbal beskrivning av ett problem, via dess tillståndsgraf, till en *realisering* med *binära* moduler (komponenter). Vid realiseringen utnyttjar vi endast 0-or och 1-or!

Modulerna kan delas upp i två klasser:

- *rena fördröjningar*, s k *d-element* (delay).

Dessa definieras av ekv. (2.21),

$$u(t) = x(t - 1) \quad (2.21)$$

där $u(t)$ är utsignalen vid tiden t och $x(t - 1)$ är insignalen vid föregående tidpunkt.

- fördröjningsfria *switchfunktioner*, som definieras av tabeller.

Vi begränsar oss i detta kapitel till modulerna (komponenterna, grindarna): NOT, AND, OR, Modulo 2 adder. Detta är ingen begränsning ur teoretisk synvinkel eftersom vi kan realisera *alla* tänkbara switchfunktioner, dvs alla tänkbara samband mellan binära insignaler vid en given tidpunkt och binära utsignaler vid samma tidpunkt, med enbart

exempelvis NOT, AND och OR grindar. (Detta kommer att visas i Kap. 4.)

I Exempel 2.11 visar vi hur vi kan sätta samman våra grindar till ett s k *kombinationsnät* (eller, med en äldre beteckning, *kombinatoriskt nät*).

Avsnittet (och kapitlet) avslutas med att vi med två exempel (Ex. 2.13 och 2.14) visar hur tillståndsgraferna i tidigare exempel (Ex. 2.2 och 2.3) kan realiseras med sekvensnät. Det är viktigt att du lär dig principen!

Om du tillgodogör dig innehållet i detta kapitel på ett "rimligt" (vad det nu är?) sätt, så kommer du inte att ha problem med att klara tentamen!

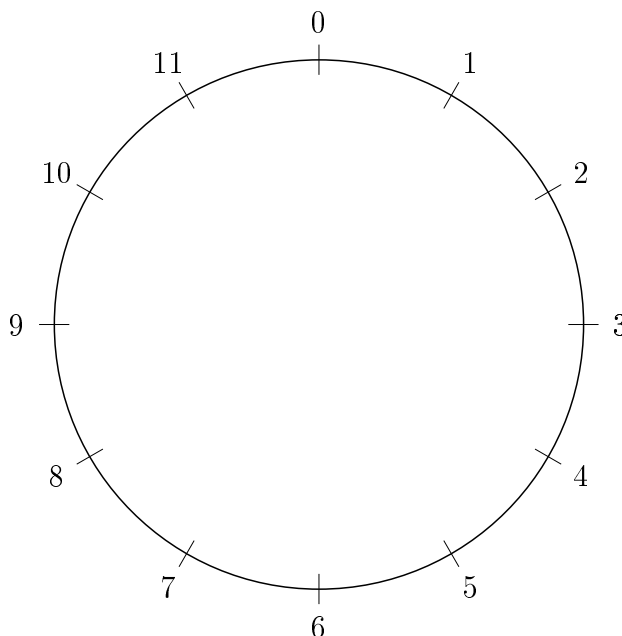
I den första laborationen kommer du att själv koppla upp enkla sekvensnät och verifiera att de fungerar som du tänkt dig!

Kapitel 3

Varför Algebra?

I detta kapitel studerar vi algebra och diskret matematik. Dessa matematikområden får en allt större betydelse inte minst inom informationsteknologi och telekommunikation. De delar av algebran och den diskreta matematiken som vi behöver inom ingenjörssämnena får nog anses vara betydligt enklare än motsvarande matematiska analys, men detta förhållande uppvägs till viss del av att de förra områdena har fått ett mycket begränsat utrymme inom ingenjörsutbildningens matematikundervisning i Sverige. De begrepp vi tar upp och diskuterar i kapitlet går något utöver vad som med ett snävt synsätt krävs i en digitalteknikkurs, men svarar väl mot vad en D- och E-civilingenjör bör ha läst i sin utbildning.

Modulo-räkning dyker upp i många sammanhang. Vi utför dagligen modulo-räkning utan att fundera så mycket på det. Klockan är nämligen ett exempel på modulo 12 räkning. Låt oss kalla klockan 12 för klockan 0. Då får vi följande urtavla:



Då vi räknar timmar rör vi oss runt cirkeln.

Om klockan är 10 och vi vill veta vad den är 7 timmar senare, så startar vi vid 10:an och går 7 steg medurs och hamnar på 5:an, dvs kl 5. Alternativt,

$$10 + 7 = 17 = 5 + 12 \quad (1)$$

dvs vi får 5 plus en multipel av 12.

Om klockan är 6 och vi vill veta vad den är 32 timmar senare kan vi starta vid 6:an och gå 32 steg medurs eller, enklare, utföra beräkningen

$$6 + 32 = 38 = 2 + 3 \cdot 12 \quad (2)$$

Vi hamnar alltså vid 2:an, kl 2.

Uträkningarna (1) och (2) är enkla tillämpningar av Euklides divisionssats (Theorem 3.1 på sid 75). Vi dividerar talet $n = 38$ med *divisorn* $d = 12$ och erhåller (*kvoten* $q = 3$ samt) *resten* $r = 2$ (jämför (3.1)).

Vi inför beteckningen $R_d(n)$ för resten r som vi får då vi delar talet n med divisorn d (se (3.2) sid 76), dvs

$$R_{12}(17) = 5$$

och

$$R_{12}(38) = 2$$

Då vi infört division av heltal är steget inte långt till *största gemensamma delaren* (*greatest common divisor*, gcd) (sid 78).

På sidorna 78–85 diskuteras och exemplifieras den största gemensamma delen och vi visar hur den kan bestämmas med Euklides algoritm. Detta är viktigt. Den största gemensamma delaren och därmed Euklides algoritm utnyttjas i många ingenjörstillämpningar.

Motiverande övning: Precis kl 1 (dag som natt) skänker man vid Kårhuset bort biljetter till Roskildefestivalen. Antag att du *måste* gå till Kårhuset första gången kl 0 (kl 12) och därefter återkomma var i :e timme där i är ett heltal $\neq 1$. Med hur långa intervall skall du besöka Kårhuset för att tillhöra de lyckligt lottade? Observera att lösningen ej är entydig. Kan vi på förhand veta vilka intervall som fungerar?

Låt oss återvända till klockan och introducera symbolen $\oplus$ för modulo-addition, dvs (3.64); $a \oplus b$ är resten vi erhåller då vi dividerar ”vanliga” summan $a + b$ med 12 (i fallet klockan). Exempelvis,

$$10 \oplus 7 = 5$$

och

$$6 \oplus 32 = 2$$

På motsvarande sätt introduceras symbolen $\otimes$ för modulo-multiplikation (3.65). Då vi räknar modulo 12 får vi exempelvis

$$10 \otimes 7 = 10$$

och

$$6 \otimes 21 = 6$$

Ringar

I allmänna fallet har vi inte 12 tal (timmar) utan m icke-negativa tal $0, 1, 2, \dots, m - 1$. Denna mängd betecknas $\mathbb{Z}_m$ och utgör tillsammans med modulo m -addition och modulo m -multiplikation ett exempel på en algebraisk struktur som kallas *ring*. Vi kan utföra de tre räknesätten *addition*, *subtraktion* och *multiplikation* i en ring.

(Egentligen är subtraktion inget räknesätt utan kan ses som addition av ett negativt tal. I ringen $(\mathbb{Z}_m, \oplus, \otimes)$ är subtraktion med a detsamma som addition med det icke-negativa talet $m - a$. Tänk på att a är ett element i mängden $\mathbb{Z}_m = \{0, 1, 2, \dots, m - 1\}$ som alltså inte innehåller några negativa tal.) Tänk dig att klockan är 3 och du vill veta vad klockan var 5 timmar tidigare. Då får du

$$\begin{aligned} 3 - 5 &= 3 + (12 - 5) \\ &= 3 + 7 = 10 \end{aligned}$$

Alltså var klockan 10.

På sidan 89 introduceras och exemplifieras begreppet *ekvivalensrelation*. I kapitel 6 kommer vi att diskutera ekvivalenta tillstånd då vi lär oss att minimera tillståndsgrafer. Du kan hoppa över ekvivalensrelationer vid första genomläsningen.

På sidan 90 ger vi en formell definition av en ring. Definitionen följs av exempel samt några egenskaper formulerade som teorem.

Ett element a (i en ring) som kan *inverteras* kallas *unit* (se Definition 3.4).

Om alla element $\neq 0$ i ringen är inverterbara (units) kallas ringen för en *kropp* (eng. *field*) (se Definition 3.5). I en kropp kan vi räkna med alla fyra räknesätten: addition, subtraktion, multiplikation samt division med tal $\neq 0$.

(På samma sätt som subtraktion egentligen inte är ett räknesätt, är inte heller division ett räknesätt utan *division med elementet a ($\neq 0$)* är *multiplikation med det inverterade elementet a^{-1}* .)

Kroppar

Följdsats 3.11 (Corollary 3.11) är viktig! Lär dig att $(\mathbb{Z}_m, \oplus, \otimes)$ är en kropp om och endast om m är ett primtal. ("om och endast om" innebär att "om" m är ett primtal så är $(\mathbb{Z}_m, \oplus, \otimes)$ en kropp och att om $(\mathbb{Z}_m, \oplus, \otimes)$ är en kropp så är m ett primtal; den senare delen av utsagan kan också skrivas som "endast om" m är ett primtal är $(\mathbb{Z}_m, \oplus, \otimes)$ en kropp. Vi studerar begreppet "om och endast om" mer ingående senare i kapitlet.)

I Exempel 3.21 visas bl a multiplikationstabellen för kroppen $(\mathbb{Z}_5, \oplus, \otimes)$. Motsvarande tabell för ringen $(\mathbb{Z}_4, \oplus, \otimes)$ är

$\otimes$	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	0	2
3	0	3	2	1

Det finns ingen 1:a raden (kolumnen) för 2! Alltså är inte elementet 2 inverterbart; det har ingen multiplikativ invers. Därför är $(\mathbb{Z}_4, \oplus, \otimes)$ ingen kropp.

Sidorna 97–107 innehåller två större tillämpningsexempel. Det första handlar om felkorrigeringar och det andra om ett sköppen-nyckel kryptosystem. Dessa exempel

kommer att diskuteras kortfattat på föreläsningen men om du vill kan du vid en första genomläsning hoppa över dessa sidor.

Booleska ringar

Vi skall nu betrakta en mycket speciell ring, nämligen en ring för vilken det gäller att $a^2 = a$ för alla element i ringen. En sådan ring kallas *Boolesk ring* (se Definition 3.8) (sid 107) och utgör grundvalen för digitalteknikens matematik.

Den Booleska ringen $(\mathbb{Z}_2, \oplus, \otimes)$ är både den enklaste och den ur digitalteknisk synvinkel viktigaste ringen. (Exempel 3.31).

På sidan 110 knyter vi först ihop de *aritmetiska operationerna* $\oplus$ och $\otimes$ i den Booleska ringen $(\mathbb{Z}_2, \oplus, \otimes)$ med de *logiska operationerna* AND, OR och NOT (eller $\wedge$, $\vee$ och $'$) och erhåller sambanden

<i>Booleska operationer</i>	<i>aritmetiska operationer</i>	
$a \wedge b$	$= ab$	(3)
$a \vee b$	$= a \oplus b \oplus (ab)$	(4)
a'	$= 1 \oplus a$	(5)

Exempel: Uttryck

$$(a \wedge b') \vee (a' \wedge b)$$

med de aritmetiska operationerna $\oplus$ och $\otimes$. Vi får först (utnyttja (5))

$$a \wedge b' = a(1 \oplus b) = a \oplus ab$$

och

$$a' \wedge b = (1 \oplus a)b = b \oplus ab$$

Alltså

$$\begin{aligned}
 (a \wedge b') \vee (a' \wedge b) &= (a \oplus ab) \oplus (b \oplus ab) \oplus ((a \oplus ab)(b \oplus ab)) \\
 &= a \oplus b \oplus \underbrace{ab \oplus ab}_{=0} \oplus (ab \oplus \underbrace{a^2b}_{=ab} \oplus \underbrace{ab^2}_{=ab} \oplus \underbrace{a^2b^2}_{ab}) \\
 &= a \oplus b \oplus 0 \\
 &= a \oplus b
 \end{aligned}
 \tag{6}$$

Observera att $ab \oplus ab = 0$, för alla värden på a och b !

Vi drar nu slutsatsen att

<i>aritmetiska operationer</i>	<i>Booleska operationer</i>	
ab	$= a \wedge b$	(7)
$a \oplus b$	$= (a \wedge b') \vee (a' \wedge b)$	(8)

där (7) och (8) är samma sak som (3) respektive (6).

Dessa samband har vi generaliserat till en allmän Boolesk ring i uttrycken (3.165)–(3.169).

Då vi använder de logiska operationerna ($\wedge, \vee, '$) talar vi om en *Boolesk algebra* (se Definition 3.9).

Då vi använder de aritmetiska operationerna ($+, \cdot$) eller, om vi vill framhålla modulo-räkning, ($\oplus, \otimes$) talar vi om en *Boolesk ring*.

Utnyttjar vi sambanden mellan de logiska och aritmetiska operationerna kan vi översätta en Boolesk algebra till en Boolesk ring och vice versa.

Studera villkoren (3.175)–(3.184). De är viktiga!

Övertyga dig om att ”manipulationsreglerna” i Theorem 3.19 och 3.20 gäller.

I avsnitt 3.4 Beginning logic är begreppen *implikation* och *ekvivalens* viktiga (sidorna 116 och 117 samt Exempel 3.38). Du kan hoppa över härledningsreglerna på sidorna 118–120.

Sammanställning av nya begrepp och dess beteckningar:

modulo-addition	$\oplus$
modulo-multiplikation	$\otimes$
divisor	d
kvot	q
rest	r
största gemensamma delaren	
till heltalen n_1 och n_2	$\gcd(n_1, n_2)$
ringen av heltal modulo m	$\mathbb{Z}_m$
inverterbart element (unit)	
kropp	F
Boolesk ring	
logiska operationer	$\wedge, \vee, ' $
Boolesk algebra	
implikation	$\Rightarrow$
ekvivalens	$\Leftrightarrow$

Kapitel 4

I detta kapitel skall vi skaffa oss fördjupade kunskaper om Booleska funktioner. Först inför vi begreppen *kubfunktion* och *kub*. En kubfunktion är en produkt av variabler som kan förekomma antingen som x_i eller x'_i . Observera att om alla variablerna ingår så får vi en kubfunktion som kan representeras med ett vertex, dvs ett hörn (en 0-dimensionell kub) i en n -dimensionell kub (dvs i det n -dimensionella binära rummet) (se Fig. 4.1 där $n = 3$). Ingår alla variabler utom en, så får vi en kubfunktion som kan representeras av en kant (en 1-dimensionell kub) i en n -dimensionell kub. Alla variabler utom två bildar en yta (en 2-dimensionell kub) i en n -dimensionell kub etc. Studera Exempel 4.3–4.7 så att du får en viss känsla för kubfunktioner och tillhörande kuber.

En godtycklig Boolesk funktion kan byggas upp som en $\vee$ -summa av kubfunktioner och alltså representeras med en union av motsvarande kuber.

Kubfunktioner och kuber är ett modernt och konsekvent sätt att illustrera Booleska funktioner. De bidrar till en intuitiv förståelse.

I avsnitt 4.2 inför vi först det viktiga begreppet *implikator* till den Booleska funktionen $f(\mathbf{x})$. Den betecknar en kubfunktion eller kub som är innesluten i $f(\mathbf{x})$. För alla kombinationer av variablerna som tillhör kuben är $f(\mathbf{x}) = 1$.

De speciella implikatorerna som svarar mot vertex, dvs hörn i det n -dimensionella binära rummet, kallas *mintermer*. Kubfunktionen för en minterm är produkten av *samtliga* variabler, där en variabel antingen ingår direkt som x_i eller som komplementet x'_i . Det är intuitivt klart att vi kan skriva en godtycklig Boolesk funktion som en disjunktion ($\vee$ -summa) av mintermerna för de vertex som ingår i funktionen. Detta uttryck kallas *disjunktiv normalform* (se Definition 4.13).

I Kapitel 2 realiserade vi några Booleska funktioner genom att utnyttja en AND-grind för varje rad i funktionstabellen som svarade mot $f(\mathbf{x}) = 1$ och därefter koppla ihop AND-grindarnas utgångar via en OR-grind. Jämför (2.28) och Fig. 2.36 i Exempel 2.11; (2.31) och Fig. 2.40 i Exempel 2.13; samt (2.33), (2.34) och Fig. 2.41 i Exempel 2.14. Dessa Booleska funktionsuttryck är skrivna på disjunktiv normalform och varje term (svarande mot en rad i funktionstabellerna och alltså mot en AND-grind) är en minterm!

Eftersom varje Boolesk funktion kan uttryckas på disjunktiv normalform drar vi den viktiga slutsatsen att

Varje Boolesk funktion kan realiseras med enbart AND-, OR- och NOT-grindar!

Mintermerna svarar mot de hörn i den n -dimensionella kuben för vilka den Booleska funktionen $f(\mathbf{x})$ av de n variablerna (insignalerna) $x_1, x_2, \dots, x_n$ är 1. Om vi i stället intresserar oss för de hörn för vilka $f(\mathbf{x}) = 0$ får vi *maxtermer*. En maxterm är $= 0$ för precis en kombination, där de n variablerna $x_1, x_2, \dots, x_n$ samtliga ingår som antingen x_i eller x'_i . En godtycklig Boolesk funktion kan skrivas som produkten av dess maxtermer. Detta uttryck kallas *konjunktiv normalform*.

Maxtermer och konjunktiva former uppfattas av de flesta som något svårare än mintermer och disjunktiva former. Pga symmetrin mellan $\vee$ och $\wedge$ samt 0 och 1 hos den Booleska algebran (dualitetsprincipen; Theorem 3.18, sid 112) borde det inte vara någon skillnad i svårighetsgrad. Problemet är troligen att vi undermedvetet tänker oss samma prioritetsordning som för addition och multiplikation.

Ur den konjunktiva normalformen drar vi slutsatsen att varje Boolesk funktion kan skrivas som en konjunktion ($\wedge$ -produkt; AND-grind) av disjunktioner ($\vee$ -summer; OR-grindar), där varje disjunktion svarar mot en maxterm.

De disjunktiva och konjunktiva normalformerna behandlas i alla digitalteknikböcker med undantag för de mest elementära. Vår tredje normalform, *ring-summe-expansion*, ser man sällan beskriven i dessa sammanhang. Detta är olyckligt eftersom den är viktig bl a då vi vill göra algebraiska manipulationer, t ex lösa ekvationssystem, med Booleska funktioner. Den dyker allt oftare upp i olika tillämpningar och är viktig vid konstruktion av en viss typ av kryptosystem, realiseringar av system för snabb aritmetik etc.

Studera Exempel 4.13 så får du en känsla för vad ring-summe-expansionen går ut på!

Läs nedre delen av sid 151 med eftertanke! Det är viktigt att du förstår att produkten av två mintermer = 0! Detta leder direkt till Theorem 4.7 som innebär att om vi realiserat en godtycklig Boolesk funktion på disjunktiv normalform, dvs som en disjunktion (OR-grind) av mintermer (AND-grindar) så kan vi byta ut OR-grinden mot en modulo 2 adderare och erhålla en alternativ realisering av samma Booleska funktion!

I avsnitt 4.3 skall du lära dig idéerna bakom NBCD och Gray koderna, som bl a användes i Kapitel 6 för tillståndskodning.

Avsnitt 4.4 kan du hoppa över vid första genomläsningen bortsett från Exempel 4.21 som beskriver en sk heladderare. Det är viktigt att du förstår detta byggblock som i avsnitt 4.5 sätts samman till en adderare för 4-bitars tal.

Två-komplement-representationen är viktig. Studera Fig. 4.13 och försök *förstå* varför två-komplement fungerar!

Viktiga begrepp

kubfunktion

kub

minterm

disjunktiv normalform

maxterm

konjunktiv normalform

ring-summe-expansion

två-komplement-representation

Kapitel 5

Detta kapitel ägnas nästan helt åt förenklingar av Booleska funktioner. Hur skall vi skriva en Boolesk funktion för att dess realisering skall kräva så liten kiselyta eller så få komponenter som möjligt? Detta är ett mycket svårt problem som emellertid förenklas något om vi begränsar oss till två-nivåers kretsar, exempelvis ett antal AND-grindar som kopplas samman med en OR-grind (disjunktiv form) eller ett antal OR-grindar som kopplas samman med en AND-grind (konjunktiv form).

I Exempel 5.2 visar vi metoden med *iterativ konsensus*. Den är inte så viktig, dvs den är inte central i kursen, men visar ett metodiskt tillvägagångssätt för att med algebraiska metoder beräkna *alla* primimplikatorer.

Förenklingen av en Boolesk funktion $f(\mathbf{x})$ bygger på att vi skall bestämma en mängd $\mathcal{C}$ av implikatorer till $f(\mathbf{x})$ sådan att $\mathcal{C}$ omfattar alla vertex i ON-mängden, dvs alla variabel-(insignal-)kombinationer i ON-mängden täcks av $\mathcal{C}$, och att $\mathcal{C}$ inte täcker några insignalkombinationer i OFF-mängden. Det senare innebär att $\mathcal{C}$ täcks av unionen av ON-mängden och DC-(Don't care-)mängden. DC-mängden består av de variabelkombinationer för vilken vi kan låta $f(\mathbf{x})$ vara antingen 1 eller 0. Det spelar ingen roll vilket, eftersom antingen uppträder inte kombinationen i praktiken eller så är det oväsentligt vad $f(\mathbf{x})$ är för denna variabelkombination. Det är viktigt att du förstår Don't care-begreppet eftersom det ofta kan utnyttjas så att vi kan uppnå avsevärda förenklingar av funktionsuttrycken!

Om vi täcker en Boolesk funktion med så få implikatorer som möjligt, så har vi en *minimal täckning* (*minimum cover*; Definition 5.1). Se Exempel 5.3 och Fig. 5.2.

Kuben, som representerar en *primimplikator*, är "så stor som möjligt", dvs den täcks inte av någon annan implikator. Detta innebär att motsvarande kubfunktion består av så få variabler som möjligt, dvs den är så enkel som möjligt, vilket är anledningen till att vi intresserar oss för primimplikatorer. Se Exempel 5.4 och Fig. 5.3.

En primimplikator säges vara *väsentlig* om den *måste* ingå i en minimal täckning, dvs om den innehåller ett vertex (minterm) som *inte* ingår i någon annan primimplikator.

En Boolesk funktion täcks alltid av en disjunktion ($\vee$ -summa) av dess primimplikatorer. Detta intuitiva resultat har vi formulerat som Theorem 5.1. Observera att detta alls icke betyder att alla primimplikatorer ingår i en minimal täckning! Vi har olika metoder för att bestämma vilka som ingår i en minimal täckning.

Enklast är att använda Karnaugh-diagram. Det är en praktisk och enkel metod som du absolut skall lära dig! Nackdelen är att om vi har fler än 5 variabler (6 variabler för den som övat mycket) så tappar vi överskådligheten och det är lätt att missa någon primimplikator. Observera att Karnaugh-diagrammen är organiserade så att intilliggande rutor skiljer sig endast i en variabel! Studera Exempel 5.5–5.10.

I praktiken har vi oftast Booleska funktioner av väsentligt fler variabler än vi kan hantera med Karnaugh-diagram. Quine-McCluskeys minimeringsmetod från 1950-talet är en klassiker som ägnats (och ägnas fortfarande) stort utrymme i de flesta läroböcker. Mikroelektronikrevolutionen har emellertid medfört att problem av väsentligt högre komplexitet än

tidigare numera hanteras rutinmässigt med hjälp av datorstöd. Det har då visat sig att Quine-McCluskeys metod inte fungerar längre. Vi har därför valt att låta den utgå ur kursen och i stället ersatt den med en modern metod (ja, så modern att den ännu inte (såvitt vi vet) finns beskriven i andra läroböcker utan endast i forskningsartiklar) som bygger på s k *signaturkuber*. Denna metod är arbetsam att genomföra för hand, men utgör basen för modernt datorstöd. Även om vår beskrivning är väsentligt enklare (tro mig!) än den som finns i forskningsartiklar, så krävs en stor arbetsinsats för att tränga genom detaljerna. Jag föreslår därför att du (åtminstone vid en första genomläsning) nöjer dig med Stefans genomgång på föreläsningen och ser avsnittet som en orientering om en modern och effektiv förenklingsmetod.

Avslutningsvis är det viktigt att du känner till standardkomponenterna som beskrivs i avsnitt 5.4.

Viktiga begrepp

- ON-mängd
- OFF-mängd
- DC-mängd (Don't care)
- minimal täckning (minimum cover)
- primimplikator
- väsentlig primimplikator

Viktigt verktyg

- Karnaugh-diagram

Kapitel 6

Minimala tillståndsgrafer

I detta kapitel återvänder vi till tillståndsgraferna. Då vi ritar en tillståndsgraf inträffar det inte så sällan att vi använder onödigt många tillstånd, vilket självfallet leder till en onödigt komplex och dyr realisering. Hur skall vi undvika detta? Eller bättre uttryckt: Hur skall vi utgående från en tillståndsgraf med onödigt många tillstånd komma fram till en med så få tillstånd som möjligt – till en *minimal* tillståndsgraf?

En icke-minimal tillståndsgraf innehåller olika tillstånd "som gör samma jobb" eller, formellt uttryckt, som är *ekvivalenta*. Att bestämma vilka tillstånd som är ekvivalenta sker stegvis. Se Algorithm RF på sid 244. En graf som inte innehåller några ekvivalenta tillstånd säges vara på *reducerad form* (Definition 6.2).

Om man inte tänker sig för är det lätt att tro att den reducerade formen är minimal. Så är det ofta men inte alltid! Antag att vi har ett visst *starttillstånd* för en graf given i reducerad form och att grafen innehåller tillstånd som vi *inte* kan nå från starttillståndet för det finns inga insignalsekvenser som för oss till dessa tillstånd. Det finns självfallet ingen anledning att ta med dessa ouppnåeliga tillstånd då vi realiserar vår graf.

Vi begränsar vårt intresse till den del av tillståndsgrafen som vi kan nå från starttillståndet s_0 . Den reducerade formen av denna del är den *minimala formen* av tillståndsgrafen för starttillståndet s_0 . Väljer vi ett annat starttillstånd får vi i allmänhet en annan minimal form! Exempel 6.2 belyser skillnaden mellan reducerad och minimal form.

Det är viktigt att du lär dig att minimera en tillståndsgraf. Proceduren är ganska enkel att lära sig och förstå, särskilt om man tänker igenom begreppet *-ekvivalens* (Definition 6.1) och förstår hur det leder till ekvivalens.

Hopcrofts metod (sid 259) kan du hoppa över vid första genomläsningen. Den har lägre komplexitet än Algorithm RF, men skillnaden saknar praktisk betydelse då du löser små exempel för hand. Om du däremot skall implementera minimeringsmetoder för datorstött konstruktion är du säkert intresserad av att kunna hantera grafer (tabeller) med många tillstånd och då kan algoritmens komplexitet vara avgörande.

Tillståndskodning

Tillståndskodning är ett knepigt problem. I slutet av 1970-talet införde vi i kursen en elegant metod för tillståndskodning som byggde på en speciell form av tillståndspartitionering (gruppering av tillstånden) – *reduced dependency*. Idén bakom denna metod är att få tillståndsvariabler ändras vid tillståndsövergångarna. (Detta var första gången som en metod för tillståndskodning förekom i undervisningen i Sverige.) Senare forskning visade emellertid att även om metoden fungerar hyfsat för små (läroboks-) exempel, t ex våra övningstal och hemuppgifter för laborationerna, så ger den inga komplexitetsvinster för realistiskt stora problem!

Vi belyser tillståndskodningsproblemet med att realisera en modulo 6 räknare på tre olika sätt (Exempel 6.5) och erhåller lösningar som är av olika komplexitet. Eftersom det inte finns några bra metoder för realistiskt stora problem har vi i nuvarande kursupplägg avsevärt minskat exercerandet med tillståndskodning och nöjer oss med att du känner till

att valet av tillståndskod påverkar realiseringens komplexitet. Då du löser dina exempel får du välja en godtycklig kod.

Mealy- och Moore-modellerna

Hittills har vi studerat *Mealy-modellen* – utsignalen beror av såväl insignal som nuvarande tillstånd. För vissa tillståndsgrafer beror utsignalen enbart av nuvarande tillstånd, dvs om vi ändrar insignalen, så händer ingenting med utsignalen förrän klocksignalen resulterar i att vi övergår till nästa tillstånd. En sådan tillståndsgraf kallas *Moore-modellen*.

Moore-modellen kan ses som ett specialfall av Mealy-modellen, men, kanske något förvånande, varje Mealy-modell kan konverteras till en Moore-modell, eventuellt till priset av att vi måste utnyttja fler tillstånd. Se Exempel 6.8. Mealy-Moore-begreppen är viktiga.

Asynkrona sekvensnät

I alla våra realiseringar har vi hittills låtit tillståndsövergångarna styras av en klocksignal, annars vet vi ju inte hur vi rusar runt i grafen. Det finns emellertid en klass av tillståndsgrafer för vilka vi lagt kraftiga restriktioner på vilka tillståndsövergångar som är tillåtna. För varje insignal kräver vi att vi skall hamna i ett *stabilt tillstånd* (Definition 6.7). Sådana tillståndsgrafer kallas *asynkront realiserbara* och kan realiseras utan *-element* och alltså utan klocksignal. Vi får ett *asynkront sekvensnät* som alltså är ett sekvensnät uppbyggt av enbart kombinationskretsar. En anledning till att vi studerar asynkrona sekvensnät är att vi skall förstå realiseringen av *-element* (se Exempel 6.14).

För att garantera att den asynkrona realiseringen skall fungera oavsett små variationer hos de ingående komponenterna krävs att tillståndsgrafen kodas *kapplöpningsfritt* (se sid 281) och att kombinationsnätet realiseras *hasardfritt*, dvs på fullständig primimplikatorform (se avsnitt 5.3). Det är mycket viktigt att du kan skilja på synkrona (vanliga) och asynkrona sekvensnät.

Viktiga begrepp

- ekvivalenta tillstånd
- ekvivalenta tillstånd
- reducerad form
- uppnåeligt tillstånd
- starkt förbunden graf
- minimal form
- tillståndskodning
- naturlig kod
- reducerat beroende-kod
- pragmatisk kod (Graykod)
- Mealy-modellen
- Moore-modellen
- asynkront sekvensnät
- stabilt tillstånd
- kapplöpningsfri tillståndskod
- hasardfri realisering
- fördröjningsmodul (latch)

Kapitel 7

I detta kapitel diskuterar vi lineära sekvensnät. De utgör en mycket viktig delmängd av de allmänna sekvensnät som vi diskuterat hittills. Pga lineariteten kan vi använda en kraftfullare arsenal av matematiska verktyg. Lineära sekvensnät dyker upp som byggblock inom en lång rad viktiga tillämpningsområden som mobiltelefoni, CD-spelare, höghastighetsmodem, kryptosystem etc.

Linearitet

Först studerar vi *lineära Booleska funktioner*. De är specialfall av *affina Booleska funktioner*. En affin Boolesk funktion kan skrivas

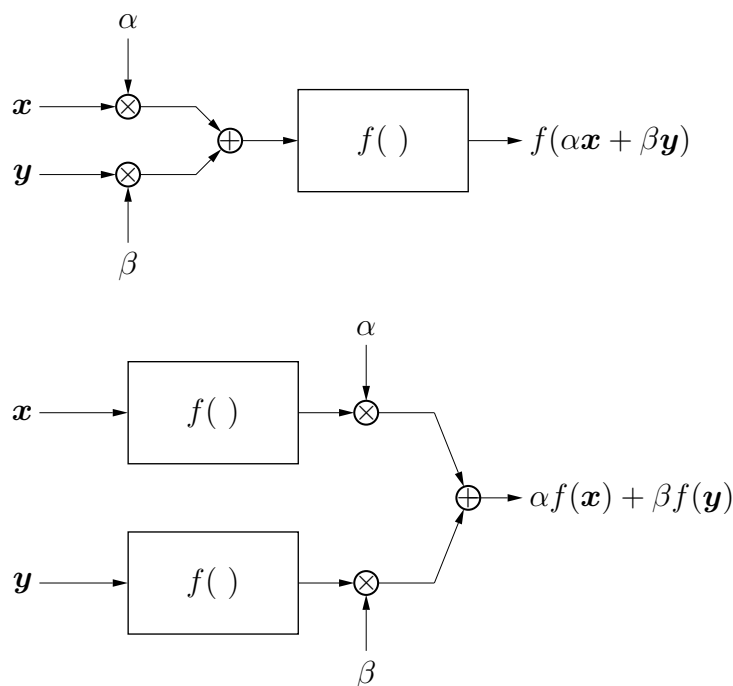
$$f(\mathbf{x}) = a_0 \oplus a_1 x_1 \oplus \cdots \oplus a_n x_n \quad (7.2)$$

och en affin Boolesk funktion är lineär om och endast om $a_0 = 0$ i (7.2). Se Theorem 7.1.

Lineära Booleska funktioner bevarar vektoraddition och skalärmultiplikation, dvs

$$L \quad f(\alpha \mathbf{x} + \beta \mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y}) \quad (7.12)$$

Linearitetsvillkoret L illustreras av nedanstående figur:



Linearitetsvillkoret formuleras ibland som att *superpositionsprincipen* gäller. Superpositionsprincipen innebär att utsignalen (värdet av funktionen) för en viktad summa av insignaler (variabler), dvs $f(\alpha \mathbf{x} + \beta \mathbf{y})$, är lika med den viktade summan av de utsignaler som erhålles då insignalerna agerar separat, dvs $\alpha f(\mathbf{x}) + \beta f(\mathbf{y})$. Superpositionsprincipen används ofta i samband med "vanliga" analoga lineära nät, dvs nät som består av motstånd, spolar och kondensatorer! Det är viktigt att du förstår linearitetsvillkoret (superpositionsprincipen).

Lineära sekvensnät

Lineära sekvensnät beskrivs oftast i matrisform:

$$\mathbf{q}^+ = A\mathbf{q} \oplus B\mathbf{x} \quad (7.21)$$

$$\mathbf{u} = C\mathbf{q} \oplus H\mathbf{x} \quad (7.22)$$

där $\mathbf{x}$ är insignalvektorn, $\mathbf{q}$ är tillståndsvektorn, $\mathbf{q}^+$ är nästa-tillståndsvektorn och $\mathbf{u}$ är utsignalvektorn.

Detta skrivsätt kommer du även att träffa på i reglerteknik och signalbehandling.

I kapitel 6 studerade vi hur en allmän tillståndsgraf kunde minimeras. Om sekvensnätet är *lineärt*, så finns det bättre metoder för minimering. En sådan beskrivs i Theorem 7.4 och illustreras i Exempel 7.2.

Eventuellt tycker du att den lineära metoden är krångligare än den generella metoden. Den lineära metoden har väsentligt lägre komplexitet men det ger knappast utslag för ”papper och penna” exempel. Det är viktigt att du känner till metoden.

Avsnitt 7.2 avslutas med en genomgång av *styrbar (controller) kanonisk form* och *observerbar kanonisk form* av ett lineärt system med en insignal och en utsignal. Observera att Fig. 7.4 och 7.5 visar dessa båda realiseringar av samma rationella (kvoten mellan två polynom) funktion

$$g(D) = \frac{b(D)}{c(D)} = \frac{b_0 \oplus b_1 D \oplus \cdots \oplus b_m D^m}{1 \oplus c_1 D \oplus \cdots \oplus c_m D^m} \quad (7.86)$$

där D svarar mot en tidsenhetsfördröjning.

Lineärt återkopplade skiftregister

Avsnitt 7.3 ägnas åt *lineärt återkopplade skiftregister* (LFSR). De kallas skiftregister eftersom fördröjningselementen är ordnade i en kedja och bildar ett register vars innehåll skiftas ett steg mot utsignalen för varje klockpuls. Eftersom återkopplingsnätet enbart består av modulo 2 grindar är återkopplingsfunktionen lineär. I Exempel 7.3 och 7.4 studerar vi två olika återkopplingsfunktioner och konstaterar att beteendet beror i allra högsta grad på återkopplingen.

Ofta beskriver vi återkopplingen med ett polynom, nämligen *återkopplingspolynomet*

$$\begin{aligned} C(D) &= 1 \oplus c_1 D \oplus \cdots \oplus c_L D^L \\ &= \sum_{j=0}^L c_j D^j \end{aligned} \quad (7.90)$$

Startar vi LFSR vid tidpunkten $t = 0$ och låter utsignalen $s^N = s_0 s_1 \dots s_{N-1}$ (7.88) representeras av polynomet

$$S(D) = s_0 \oplus s_1 D \oplus \cdots \oplus s_{N-1} D^{N-1}$$

så gäller sambandet (Theorem 7.5):

$$C(D)S(D) = P(D) \quad (\text{modulo } D^N) \quad (7.92)$$

där gradtalet för $P(D)$, $\deg P(D)$, är mindre än gradtalet för $C(D)$, dvs längden av registret, L .

Sambandet (7.92) är viktigt och leder till Theorem 7.6.

Vi inför *perioden av en sekvens* som är det minsta positiva heltal T sådant att $s_{i+T} = s_i$ för alla i , samt *perioden av ett polynom $C(D)$* som är det minsta positiva heltal T sådant att $C(D)$ delar $1 \oplus D^T$, dvs sådant att $C(D)$ är en faktor i $1 \oplus D^T$.

Om $C(D)$ är återkopplingspolynomet för ett LFSR, så är perioden av utsekvensen lika med perioden av $C(D)$!! Viktigt resultat, som finns formulerat som Theorem 7.7.

Algoritmer för studium av skiftregister

Lär dig Euklides algoritm för polynom (sid 329). Med den kan du på samma sätt som du i Kapitel 3 bestämde största gemensamma delaren till två heltal enkelt och snabbt bestämma största gemensamma delaren till två polynom. Detta utnyttjar vi då vi skall bestämma det kortaste LFSR som genererar en given periodisk sekvens!

Antag att den periodiska sekvensen är $[s_0 s_1 \dots s_{T-1}]^\infty$ (sid 326). Dess $\mathcal{D}$ -transform ges av

$$\mathcal{D}([s_0 s_1 \dots s_{T-1}]^\infty) = \frac{s_0 \oplus s_1 D \oplus \dots \oplus s_{T-1} D^{T-1}}{1 \oplus D^T} \quad (7.104)$$

Med hjälp av Euklides algoritm bestämmer vi den största gemensamma delaren till täljarpolynomet $s_0 \oplus s_1 D \oplus \dots \oplus s_{T-1} D^{T-1}$ och nämnarpolynomet $1 \oplus D^T$. Denna sgd ingår som en faktor i båda polynomen och kan således divideras bort. Då får vi det nya nämnarpolynomet

$$C(D) = \frac{1 \oplus D^T}{\text{sgd}(s_0 \oplus s_1 D \oplus \dots \oplus s_{T-1} D^{T-1}, 1 \oplus D^T)}$$

som är återkopplingspolynomet för det kortaste LFSR som genererar den *periodiska* sekvensen $[s_0 s_1 \dots s_{T-1}]^\infty$! Proceduren illustreras av Exempel 7.8 och är mycket viktig!

Med hjälp av Berlekamp-Masseys algoritm (Fig. 7.14) kan vi enkelt bestämma det kortaste LFSR som genererar (den *aperiodiska*) sekvensen $s_0 s_1 \dots s_{N-1}$. Räkna några exempel och bekanta dig med denna listiga algorit!

Skilj på dessa båda fall! Den periodiska sekvensen är *oändlig* medan den aperiodiska sekvensen är *ändlig*!

Längden av det kortaste LFSR som genererar en ändlig sekvens kallas sekvensens *lineära komplexitet*, medan längden av det kortaste LFSR som genererar en (oändlig) periodisk sekvens kallas sekvensens *cykliska komplexitet*.

Olika sätt att generera sekvenser

I Fig. 7.11 visar vi ett lineärt sekvensnät som är av speciellt intresse. Det är *inte* ett LFSR, men väl *ekvivalent* med ett LFSR, nämligen det LFSR som illustreras i Fig. 7.8.

Jämför dessa båda kretsar och observera särskilt att återkopplingspolynomen går ”åt olika håll”! Med lämpliga starttillstånd, dvs sekvensens L första symboler i Fig. 7.8 och täljarpolynomet $P(D)$ i Fig. 7.11, erhålles identiska utsekvenser. Valet av realisering är en smaksak; vid hårdvarurealisering föredrar man oftast Fig. 7.8 medan man vid programvarurealisering oftast väljer Fig. 7.11. Lär dig skillnaden mellan Fig. 7.8 och Fig. 7.11.

Avsnitten 7.4 och 7.5 kan du hoppa över.

Anmärkning. Lineära sekvensnät och särskilt lineärt återkopplade skiftregister är såväl praktiskt viktiga som teoretiskt intressanta. Vi kan härleda många viktiga resultat för dem. I kommande versioner kommer kapitlet att ökas ut något och avsnitten 7.4 och 7.5 att bearbetas.

Viktiga begrepp

- affin Boolesk funktion
- lineär Boolesk funktion
- linearitetsvillkoret
- superpositionsprincipen (saknas i kapitlet, men beskrivs på sid 18 i dessa läsanvisningar)
- matrisformen för lineära sekvensnät
- styrbar (controller) kanonisk form
- observerbar kanonisk form
- lineärt återkopplade skiftregister (LFSR)
- återkopplingspolynomet $C(D)$
- perioden av en sekvens
- perioden av ett polynom
- Euklides algoritm för polynom
- kortaste LFSR för en periodisk sekvens
- kortaste LFSR för en ändlig sekvens
- Berlekamp-Masseys algoritim
- lineär komplexitet
- cyklisk komplexitet